System Verification and Validation Plan for Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

November 2, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 2

3 Plan **2**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 6
- 3.5 Implementation Verification . . . 7
- 3.6 Automated Testing and Verification Tools . . . 8
- 3.7 Software Validation . . . 9

4 System Tests **10**
- 4.1 Tests for Functional Requirements . . . 10
  - 4.1.1 Area of Testing1 . . . 10
  - 4.1.2 Area of Testing2 . . . 11
- 4.2 Tests for Nonfunctional Requirements . . . 11
  - 4.2.1 Area of Testing1 . . . 12
  - 4.2.2 Area of Testing2 . . . 12
- 4.3 Traceability Between Test Cases and Requirements . . . 13

5 Unit Test Description **13**
- 5.1 Unit Testing Scope . . . 13
- 5.2 Tests for Functional Requirements . . . 13
  - 5.2.1 Module 1 . . . 13
  - 5.2.2 Module 2 . . . 14
- 5.3 Tests for Nonfunctional Requirements . . . 15
  - 5.3.1 Module ? . . . 15
  - 5.3.2 Module ? . . . 15
- 5.4 Traceability Between Test Cases and Modules . . . 15

6 Appendix **18**
6.1 Symbolic Parameters 18
6.2 Usability Survey Questions? 18

7 Appendix — Generative AI Use Disclosure **19**
7.1 AI Tools Used 19
7.2 How and Where Used 19
7.3 Verification of AI-Generated Content 19

8 Appendix — Reflection **21**

List of Tables

2 Lake Verification and Validation Team 4
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

Term	Description
Activation	An intermediate value that a model layer computes and passes to the next layer.
AI	Artificial intelligence. Software that does tasks that normally need human judgement. In this document it always means an LLM.
API	Application programming interface. A set of requests one program can make of another. Lake's API is the OpenAI-compatible API on the leader node.
ARM	One of the two CPU architectures Lake supports, used by Apple silicon and most mobile devices.
Backpressure	A signal that a node is receiving tensors faster than it can process them, so the upstream node should slow down.
Bottleneck	The node or network connection that takes the longest per token and limits the throughput of the whole Lake network.
Context window	The maximum number of tokens, prompt and response together, that a model can handle in one request.
Continuous integration	Building and testing the code automatically on every push to the repository.
Coordinator	The Lake component that forms and monitors the Lake network, serves the API on the leader node, and manages the Inference Engine on each node.
CPU	Central processing unit. A device's general-purpose processor.
Device owner	The person whose computer joins a Lake network. They must consent before their device takes part.

Term	Description
Dynamic allocation	Moving model layers between nodes while the Lake network is running, when a node joins, leaves, or fails.
FFI	Foreign function interface. An interface that lets code in one language call code in another.
FLOP/s	Floating-point operations per second. The number of floating-point operations a processor performs per second, used to measure compute performance.
Follower node	Any node in a Lake network other than the leader node.
GGUF	The file format used by llama.cpp and ggml to store a model's weights, tokeniser, and metadata. Lake only accepts GGUF models.
GPL	GNU General Public License. The free software licence, version 3.0, that Lake is released under.
GPU	Graphics processing unit. A device's parallel processor, which does the arithmetic of inference.
Greedy decoding	Always choosing the most probable next token, so the same prompt gives the same output every time.
gRPC	A remote procedure call framework from Google. Tensor I/O uses it to send tensors between nodes.
HTTP	Hypertext Transfer Protocol. The request and response protocol of the web.
Inference	Using a model to generate text from a prompt.
Inference Engine	The Lake component on each node that loads the node's partition and runs it to generate output. It is built on ggml.
Inference user	A person, or a tool acting for one, who sends prompts to a Lake network and gets responses back.
IPC	Inter-process communication. Passing data between processes on the same machine.

Term	Description
KV cache	The key and value tensors each layer stores for past tokens so they are not recomputed. Each node keeps the KV cache for its own layers.
Lake	The system this document specifies. It runs one LLM across several devices on a local network using model layer partitioning.
Lake Client	The Lake application installed on each device, and the component people use. Every node runs one. The operator forms a Lake network, loads a model, and views performance from the Lake Client on the leader node, and a device owner confirms joining from the Lake Client on their own device.
Lake network	The nodes on one local network that have joined to serve one model.
Latency	The time between the start of an inference request and a defined event, such as the first token or the end of the response.
Layer	One of the stacked stages of a transformer model. Lake splits a model between nodes at layer boundaries.
Leader node	The node whose operator formed the Lake network. It runs the Model Partitioner, serves the API, and monitors the followers.
LLM	Large language model. An AI model trained on a large body of text and used to generate text.
Model layer partitioning	Splitting an LLM's layers into consecutive ranges and giving each range to a different node.
Model Partitioner	The Lake component on the leader node that decides which layers each node holds, based on each node's memory, compute, and network performance.

Term	Description
MoSCoW	A prioritisation scheme that sorts requirements into Must have, Should have, Could have, and Won't have.
NIST	National Institute of Standards and Technology. The United States standards agency.
Node	A device running the Lake Client that has joined a Lake network.
OCR	Optical character recognition. Converting images of text into machine-readable text.
OpenAI-compatible API	An HTTP API with the same request and response formats as the OpenAI API, so clients written for OpenAI work with Lake unchanged.
Operator	The person who forms a Lake network, loads a model, and monitors performance from the Lake Client. Also called the Lake operator.
P2P	Peer-to-peer. A direct connection between two nodes that does not go through the leader. Neighbouring nodes send tensors this way.
Partition	A consecutive range of a model's layers assigned to one node.
Partitioning scheme	The assignment of partitions to nodes for a whole model.
PEGS	Meyer's four-chapter structure for a requirements document: Project, Environment, Goals, and System.
Perplexity	A measure of how well a model predicts a body of text, where lower is better. Used to compare a split model with the unsplit one.
PIPEDA	Personal Information Protection and Electronic Documents Act. Canada's private-sector privacy law.
PoC	Proof of concept. The first working demonstration of the system.

Term	Description
Prefill	The first phase of a request, where the whole prompt runs through the model to fill the KV cache before the first output token.
Quantisation	Converting a model's weights to a lower-precision format to reduce memory use.
RACI	A chart showing who is Responsible, Accountable, Consulted, and Informed for each piece of work.
REST	Representational state transfer. A style of HTTP API where clients read and change named resources.
RPC	Remote procedure call. Calling a function on another machine as if it were local.
RTT	Round-trip time. The time for a message to reach another node and for the reply to come back.
Session	A period of interaction with Lake in which one or more inference requests are served. For the operator, it runs from forming a Lake network to shutting it down. For an inference user, it is one conversation whose context is kept between prompts.
SOLID	Acronym for specifying design principles. Stands for Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.
SRS	Software Requirements Specification. This document.
Streaming	Returning a response token by token as it is generated.
TA	Teaching assistant. The course staff member who advises the team and assesses its deliverables.
Telemetry	Usage statistics, crash reports, update checks, or any other data that software sends to its developer or a third party. Lake sends none.

Term	Description
Tensor	A multi-dimensional array of numbers. Weights, activations, and the KV cache are all tensors.
Tensor I/O	The Lake component that compresses, sends, receives, and decompresses tensors between neighbouring nodes.
Token	A unit of text processed by an LLM, typically a word or part of a word.
Token throughput	The number of tokens generated per unit time, usually tokens per second.
Tool calling	An OpenAI API feature where the model asks the client to run a named function and return the result.
Transformer	The neural network architecture of current LLMs, built from a stack of layers.
UI	User interface. The part of a program a person interacts with. Lake's UI is the Lake Client.
V&V	Verification and validation. Verification checks that the system meets its requirements. Validation checks that it is the right system for its stakeholders.
Weights	The learned parameters of a model that determine its behaviour. Lake never modifies them.
x86_64	The 64-bit instruction set of Intel and AMD processors. One of the two CPU architectures Lake supports.

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

This plan covers the core release of Lake as scoped in the SRS.

2.1 Summary

The software under test is Lake, which runs one large language model (LLM) across several consumer devices on a local network. Lake splits the model by layer and gives each device a consecutive range of layers. The operator forms a Lake network from the Lake Client, loads a GGUF model, and sends prompts through an OpenAI-compatible API. When a node joins or leaves, Lake reassigns the layers and keeps serving requests.

All five components are in scope: the Coordinator, the Model Partitioner, the Inference Engine, Tensor I/O, and the Lake Client.

2.2 Objectives

The qualities we most need confidence in are output correctness, privacy, reliability when devices come and go, and performance.

Splitting a model must not change what it produces. The split model's benchmark scores must stay within *BENCH_RATIO* of the unsplit model's. Related to NFR.S.6.

No node may keep request data after a request finishes, and nothing from a request may leave the Lake network except the response to the client. Related to NFR.S.10, NFR.S.11, and SR5-2.

For reliability, Lake must keep serving requests when a node fails, leaves, or joins, and must refuse a model that would run a node out of memory. Related to FR.S.1.10 to FR.S.1.12, SR1-3, and SR2-1.

For performance, a Lake network must run a model too large for its biggest node, and the number of requests it serves at once within *MAX_DEGRADATION* must grow with the number of nodes. Single-request speed is measured but has no pass threshold, since a split model is not expected to beat one device on a model that already fits. Related to NFR.S.4 and NFR.S.5.

Usability is checked with the survey in the appendix rather than a formal study. For platforms, we test the operating system, CPU, and GPU combinations the team owns and report the rest as untested in the V&V Report.

We test how Lake uses ggml and llama.cpp, not the libraries themselves. The content a correctly running model generates is also out of scope. The stretch goals (encryption, authentication, wireless links, and smartphones) are not tested this year, and neither are attacks from inside the local network.

2.3 Custom Documents

Our custom documents are a Design Thinking Report in the fall and a Performance Report in the winter (Chambachan et al., 2026e). Software validation checks the system against the user needs recorded in the Design Thinking Report. The Performance Report measures model size, token throughput, latency, and reassignment time beyond the SRS targets, starting from the results of the performance tests in this plan.

2.4 Relevant Documentation

Every system test traces to a requirement in the SRS (Chambachan et al., 2026f), which also defines the symbolic constants used here. The Hazard Analysis (Chambachan et al., 2026b) produced the safety requirements SR0-1 to SR5-2, and each has at least one test. Five of its 12 failure modes are in communication between nodes, so most of our fault-injection tests target that area.

The Development Plan (Chambachan et al., 2026a) sets the linters, test frameworks, and continuous integration pipeline our automated tests use. Unit tests will be written against the Module Guide (Chambachan et al., 2026c) and Module Interface Specification (Chambachan et al., 2026d) once they exist.

3 Plan

The following is a set of verification practices that the Lake team will use to ensure that requirements, design, implementation, and testing are each verified throughout the development process.

3.1 begins by establishing team roles specifically for project verification. Following this, 3.2, 3.3, and 3.4 specify how the team will verify various non-code artifacts - the Chambachan et al. (2026f), design, and the verification and validation plan (this document).

Finally, 3.5 - Implementation Verification, 3.6 - Automated Testing and Verification Tools, and 3.7 - Software Validation provide further specification on how code artifacts will be validated and verified.

3.1 Verification and Validation Team

The team will take on overall responsibility for the execution of Verification and Validation, based on the following roles and responsibilities.

Table 2: Lake Verification and Validation Team

Team Member	**Role**	**Verification and Validation Responsibilities**
Gary Qin	Lead	Oversight of the execution of the plan. Responsible for promoting changes brought forth by R&D, ensuring new tools are implemented, and that automated verification and pipeline tools are operating as intended.
Jinwoo Hong	Research & Development	Research of state of the art verification/validation tooling and providing recommendations to the team. If required, small-scale integration of new validation tools for demonstration purposes and to prove effectiveness.
Matthew Nesbitt	DevOps & CI-CD	Integration of identified tools and practices into the project's pipeline, ensuring that all code and documentation changes are checked automatically for adherence. Repair and fixes to CI-CD as required.
Colin Chambachan	Implementation	Working with Coordinator and Research & Development team members to implement tools and practices in the codebase. This applies for document- and code-based verification and validation.
Ethan Walsh	Coordinator	Ensuring that each component of the plan has been executed to the team's quality standard. Additionally, tracking updates to plan and updating documentation to reflect what is up-to-date.

3.2 SRS Verification

In order to ensure that the SRS meets expectations, and that the project meets what was originally set out in the SRS, the following verification methods will be used:

- **External Peer Review**: Since the team aims to validate their project externally and Lake has no designated supervisor, the team will seek external feedback from a peer who is not a member of the Capstone course. This peer should be familiar with the project and its goals at a high-level, possess relevant background knowledge, and can perform an ad-hoc review. This allows the team to surface new insights and improve the readability and quality of their SRS.

- **Internal Peer Review**: As with each document in the Capstone project, there is a review process performed by another team. This review aims to find consistency and content issues within the document, and will be used to provide the team additional feedback.

- **GitHub Issue Tracking**: All feedback provided to the team will be noted and converted into issues on GitHub. This allows the team to maintain a backlog of SRS-related issues and fix identified problems.

- **Internal Team Verification** (Once per deliverable): As the team progresses through the term, they will review their progress to ensure it aligns with what was set out in the SRS document. In the case new code or documentation does not align with the SRS, the team will discuss the necessary fix. This will result in either the SRS being amended to reflect the project more accurately, or the deliverable will be changed. The goal of this process is to guarantee consistency: what is stated in any code or documentation after the SRS should logically follow from what was prescribed in it.

- **SRS Checklist**: In order to complete internal team verification, the lead for verification and validation (as set out in 3.1) will create a checklist of criteria. This checklist will be used by the team to ensure each deliverable logically follows from the SRS, and if not, the necessary changes have been completed and tracked sufficiently.

3.3 Design Verification

Design verification of Lake will begin after November 16, when the first design document will be complete. In order to verify the design of Lake, the following methods will be used:

- **Peer Review**: Similar to verification of the SRS that was detailed in 3.2, peer review will be used to verify the design documentation. This will involve the traditional peer review process from another Capstone team, along with external review. External review will be used as a method to challenge the team's assumptions and point out potential design flaws.
- **Inspection of Module Guide and Module Interface Specification**: After the document has been completed, the team will perform a review of the design documentation. This will be used to identify inconsistencies, gaps between design and requirements, and flag areas of concern. Any areas of concern should be noted as GitHub issues and resolved prior to further design work.
- **Checklist-based Verification**: The team will combine results of the peer review along with an existing checklist to complete the design verification process. The checklist will focus on the following:
  - Adherence to general software design principles and practices (e.g., SOLID)
  - Adherence to modular design principles (minimal effort to build additional features)
  - Consideration for design scaling and performance-oriented non-functional requirements (ability to scale and expand the design to accommodate additional requests, devices, and traffic).

3.4 Verification and Validation Plan Verification

In order to verify the quality of the team's verification and validation plan, the team will employ the following methods:

- **Internal Review**: Throughout the preparation of the document, the team will review individual document sections and how they connect to guarantee quality and consistency.

- **Peer Review**: The team will use the peer review process to catch additional issues or inconsistencies. This will be used to complement the completed internal review.

- **Ad-Hoc Team Meeting**: The team will schedule a one-time meeting after the delivery of the VnV plan to assess its feasibility, as well as assign responsibilities to each member. This meeting will aim to assess the effectiveness and practicality of the current plan, and make changes if required.

3.5 Implementation Verification

The implementation will utilize a combination of code review, tests, benchmarks, and issue tracking methods to verify correctness and performance in an organized and auditable manner.

- **Code review:** GitHub pull requests will be created for code review following the Development Plan's process of approving code changes. At least two reviewers will check for correctness, error handling, and adequate tests for the code changes. A max of 400 lines added per pull request constraint is added to ensure each pull request is auditable for reviewers, with larger features split into several pull requests.

- **Unit and integration tests:** Unit tests will be used to ensure correctness in areas such as model layer allocation constraints, lifecycle transitions of node health, and message format validation to ensure compatibility with interfaces (e.g., the OpenAI API spec). Integration tests will be used to validate runtime behavior of running multiple node processes on a single machine with a small model to ensure functional end to end behavior. Both checks will run in the CI using GitHub runners, configured such that the relevant tests respective to the code changes are run.

- **Fault Handling:** Tests will introduce faults in lifecycle and data states to inject stale health data, unavailable nodes, corrupted or incomplete tensor messages, and communication delays. The results will be checked against the defined safety requirements in bounded recovery, explicit failures, and filtration of sensitive data in diagnostic logs.

- **System Testing:** Tests for Lake's end-to-end behavior that may require multiple devices, various supported GPU backends and operating systems, and varying degrees of system capabilities will be run on team hardware before major releases or demonstrations. System test coverage will identify the operating system, CPU architecture, GPU backends, and system capability metrics (RAM/VRAM, CPU/GPU performance). Combinations of system configurations and capabilities that are are not tested will be reported as untested in the V&V report. Each recorded result will also identify the commit of the current code version, the model as well its quantization, node configuration, and test inputs for repeatability.

- **Performance Verification:** Performance metrics such as Lake's capacity for model size, concurrent request throughput, and output quality will be used to verify that optimizations result in quantitative performance gain, and that changes do not regress performance, as specified in Section 4.2.

- **Defect Tracking:** Failed CI checks will block pull requests from being merged. If review findings are too large in scope to be addressed in the pull request, they will instead be filed as GitHub issues. Regression tests will be added when a failure is identified to prevent it from recurring.

3.6 Automated Testing and Verification Tools

The following testing and verification tools will be used in accordance with the technologies specified in the Development Plan.

- **Testing frameworks:** `pytest` (pytest developers) will be used for Python components, `cargo test` (Rust project developers) for Rust components, and `Vitest` (Vitest contributors) for TypeScript components. Tests will cover behavior around the components of the coordinator, model partitioner, inference engine, Tensor I/O, and the Lake client.

- **Static checks:** `Ruff` (Astral, 2026) will be used for Python code, `Clippy` (Rust project developers, 2026a) and `rustfmt` (Rust project developers, 2026b) will be used for Rust code, and `ESLint` (OpenJS

Foundation, 2026) and `Prettier` (Prettier contributors, 2026) will be used for TypeScript code. These checks will be run on commit hooks and pull request CI.

- **Continuous integration:** GitHub Actions (GitHub) will be used to run checks and automated tests on Linux, Windows, and macOS runners. To minimize CI minutes, applicable tests will use lightweight models across local processes. Tests requiring specific hardware that GitHub runners do not provide will instead be run on team hardware outside of the CI.

- **Code coverage:** `pytest-cov` (pytest-cov contributors) will be used for Python code coverage, and `cargo-llvm-cov` (cargo-llvm-cov contributors) will be used for Rust code coverage. Code coverage reports will be generated that summarize which components and code are not exercised by a test suite.

- **Memory and network checks:** `AddressSanitizer` (LLVM Project) will be used for verifying unsafe `ggml` (ggml-org contributors, 2026a) integrations. `tc netem` (iproute2 contributors) on Linux will be used for packet loss and delays as specified in Section 4.2.

- **Performance comparisons:** Scripts will be used to measure concurrent request throughput, as well as compare model accuracy scores against the same model served by `llama-server` (ggml-org contributors, 2026b) as defined in Section 4.2.

3.7 Software Validation

Software validation assesses if Lake meets the needs described by the SRS goals, stakeholder interview, and user profiles. In addition to checking these requirements, the team will gather feedback on whether the workflow and user experience are useful and intuitive.

- **Task-based user sessions:** Participants who have not used Lake, such as other students outside of the capstone team, will complete tasks to set up and utilize the Lake network as specified in Section 6.2. These sessions will utilize the participant count and time limits specified in Section 6.1. With the data recorded from these sessions, the

team will gauge whether users can correctly and easily configure Lake and understand how participating devices work, as well as determine the throughput and bottlenecks of the system.

- **Demonstrations of benefits:** During the PoC and revision 0 demonstration, the team will demonstrate the functionality of Lake at its current stage to collect feedback on its usefulness and benefits. Demonstrations will show a client that is connected to the Lake API to access a model via a pooled device network, and measurements across concurrent requests, model capacity, and output quality will be made, corresponding to the tests in Section 4. Participants of the Lake network at this stage can also comment on if the response times and token throughput of the model are acceptable for their use case.

- **Stakeholder interview:** Subject to availability of the previously interviewed AI developer from the SRS, the team will seek follow-up feedback. The follow-up will involve the ergonomics of integrating Lake into the company's workflow, the implementation of the security and telemetry requirements, and any limitations that would prevent operation at their company.

Feedback from the software validation will be recorded as GitHub issues and used to iterate on the implementation or revise requirements.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

Output:

How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Astral. Ruff: An extremely fast Python linter and code formatter. https://docs.astral.sh/ruff, 2026. Accessed 19 October 2026.

cargo-llvm-cov contributors. cargo-llvm-cov. https://github.com/taiki-e/cargo-llvm-cov. Accessed 2 November 2026.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Development plan for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf, 2026a.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Hazard analysis for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, 2026b.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Module guide for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, 2026c.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Module interface specification for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, 2026d.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Problem statement and goals for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/ProblemStatementAndGoals/ProblemStatement.pdf, 2026e.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Software requirements specification for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/SRS/SRS.pdf, 2026f.

ggml-org contributors. ggml: Tensor library for machine learning. https://github.com/ggml-org/ggml, 2026a. Accessed 16 October 2026.

ggml-org contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2026b. Accessed 16 October 2026.

GitHub. GitHub Actions documentation. https://docs.github.com/en/actions. Accessed 2 November 2026.

iproute2 contributors. tc-netem: Network Emulator manual. https://man7.org/linux/man-pages/man8/tc-netem.8.html. Accessed 2 November 2026.

LLVM Project. AddressSanitizer documentation. https://clang.llvm.org/docs/AddressSanitizer.html. Accessed 2 November 2026.

OpenJS Foundation. ESLint: Find and fix problems in your JavaScript code. https://eslint.org, 2026. Accessed 19 October 2026.

Prettier contributors. Prettier: An opinionated code formatter. https://prettier.io, 2026. Accessed 19 October 2026.

pytest-cov contributors. pytest-cov documentation. https://pytest-cov.readthedocs.io/en/latest/. Accessed 2 November 2026.

pytest developers. pytest documentation. https://docs.pytest.org/en/stable/. Accessed 2 November 2026.

Rust project developers. The Cargo Book: cargo test. https://doc.rust-lang.org/cargo/commands/cargo-test.html. Accessed 2 November 2026.

Rust project developers. Clippy: A collection of lints to catch common mistakes in Rust code. https://doc.rust-lang.org/clippy, 2026a. Accessed 19 October 2026.

Rust project developers. rustfmt: Format Rust code. https://github.com/rust-lang/rustfmt, 2026b. Accessed 19 October 2026.

Vitest contributors. Vitest: Getting Started. https://vitest.dev/guide/. Accessed 2 November 2026.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

ChatGPT was used to brainstorm and research ideas and tools alongside course material for Section 3.5, 3.6, 3.7. It was also used for adding template/placeholder LaTeX code as well as a final grammar check.

ChatGPT was also used for the appendix for template/placeholder LaTeX code and a final grammar check.

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Our SRS and Hazard Analysis documents that identified several functional, non-functional, and safety requirements gave us a good starting point for defining the test plan. We used these documents to work from a specific requirement to an input to a test, an expected result, and a method to check that result, without having the need to come up with a general list of tests beforehand. For example, the requirements for stale health data and bounded recovery were used to specify tests for ensuring correctness during delayed health reports or a follower node getting interrupted.

   We also divided the document by responsibility to improve parallelism and reduce blockers while our team worked. Dividing the document and reviewing things together was important as the verification plan, test cases, traceability tables and symbolic parameters all described related activities, just from different perspectives.

   Compared to the SRS and Hazard Analysis documents, we clarified which members owned the sections earlier, as well as thoroughly reviewed and discussed the drafts and pull requests before merging. This helped align the different parts of the V&V, such as test cases and symbolic parameters before merging them together.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   A difficult part of the deliverable was planning the tests for the distributed behavior of Lake (i.e. testing how Lake behaves across computers with different system configurations and hardware capabilities), before the implementation and hardware behaviour were available. We addressed this by separating lightweight automated CI tests from hardware and system tests we would run on our own hardware.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   - **Jinwoo Hong:** Evaluating coverage and memory-checking tools for Rust, especially for areas involving ggml through an unsafe foreign function interface. This is important for testing as it requires figuring out what can and cannot be determined correctness-wise in this area.
   - **Gary Qin:** Designing experiments to determine model capacity, max concurrent requests, and output quality and compare with past baselines, while also configuring various model settings and device configurations.
   - **Matthew Nesbitt:** Building CI workflows across Linux, Windows, and macOS operating systems, while separating automated CI tests from those that require real manual hardware configurations.
   - **Colin Chambachan:** Implementing various fixtures and fault injection tests that can reproduce failures in the distributed system while keeping these tests automated in the CI.
   - **Ethan Walsh:** Ensuring tests are traceable to requirements and being able to audit and inspect missed acceptance criteria, infeasible tests, and any other changes that require tests to be updated.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge

or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

- **Jinwoo Hong:**
  - **Approach 1:** Study the documentation for how to utilize the coverage and memory checking tools for Rust. Read Section 20.1 of the Rust Book to learn about calling C functions and creating safe abstractions around unsafe Rust.
  - **Approach 2:** Compare and learn the tools by working on a small example of both Rust and C communicating.
  - **Chosen approach and reason:** The best approach combines both, focusing on reading documentation and also trying a small implementation to get a better understanding of potential difficulties when implementing with these tools.
- **Gary Qin:**
  - **Approach 1:** Review how existing benchmarks work for evaluating LLMs (e.g., MMLU) and study their methodologies.
  - **Approach 2:** Build a small experiment that varies one factor at a time to easily determine how different changes affect end results.
  - **Chosen approach and reason:** The proposed approach was to run a small experiment supported by documentation and methods used in other benchmarks. This can reveal which configurations cause unstable results before full hardware runs.
- **Matthew Nesbitt:**
  - **Approach 1:** Study the documentation for GitHub Actions and how Rust and Python workflows work across Linux, Windows, and macOS, focusing on areas such as dependency installation, caching, and retaining test reports so that failed runs can be investigated.
  - **Approach 2:** Build a small workflow that compiles a component and runs a test on it on one operating system. Also introduce a failing test to see how the error information can be retrieved.

  - **Chosen approach and reason:** The second approach was chosen because it allows for incrementally understanding and building the CI workflows. Once small components are built, they can be extended to the other operating systems to check platform-specific build and test behaviour.

- **Colin Chambachan:**
  - **Approach 1:** Review various examples of test fixtures, test doubles, and any course content from SFWRENG 3S03 that may be relevant for this area.
  - **Approach 2:** Implement a small test for fault injection, such as withholding a node's health reports until they become stale, and ensuring the system behaves correctly according to the relevant requirements.
  - **Chosen approach and reason:** Approach 1 should be a prerequisite if knowledge on the material is insufficient, otherwise approach 2 was chosen as it allows for more incremental improvements and understanding of the system and tests.

- **Ethan Walsh:**
  - **Approach 1:** Review the course V&V checklist and the team's traceability table to learn how to identify and interpret missing tests or unclear results.
  - **Approach 2:** Review some requirements and tests with a teammate then compare each other's findings about any missing cases or infeasible test procedures.
  - **Chosen approach and reason:** Assuming familiarity with the course material, the second approach was chosen for helping ensure there are no overlooked results, and that it better applies to Lake's use-case.